

Software Requirements Specification (SRS)

Skillify Freelancing Platform

****Version:**** 1.0

****Date:**** April 30, 2026

****Prepared for:**** Skillify Project

1. Introduction

1.1 Purpose

This document defines the software requirements for Skillify, a freelancing platform that connects clients and freelancers in a single web application. It describes the system scope, users, functional requirements, non-functional requirements, interfaces, and system diagrams.

1.2 Scope

Skillify allows:

Users to sign in and choose a role as either a client or a freelancer.

Freelancers to create and manage professional profiles.

Clients to create and manage company profiles.

Clients to post gigs and review proposals.

Freelancers to browse gigs and submit proposals.

Both sides to communicate through chat.

Secure payment processing through Razorpay.

The system is implemented as a web application using Next.js frontend routes and Convex backend services.

1.3 Definitions, Acronyms, and Abbreviations

****SRS:**** Software Requirements Specification

****UI:**** User Interface

****API:**** Application Programming Interface

****Razorpay:**** Payment gateway used for online transactions

****Convex:**** Backend platform used for database and server actions

****Clerk:**** Authentication provider used for sign-in and user sessions

1.4 Intended Audience

This document is intended for:

Project owner

Developers

Testers

Reviewers

Future maintainers

1.5 References

Next.js documentation

Convex documentation

Clerk authentication documentation

Razorpay checkout documentation

1.6 Technology Stack

Frontend: Next.js

Backend: Convex

Authentication: Clerk

Payments: Razorpay

Language: TypeScript

2. Overall Description

2.1 Product Perspective

Skillify is a two-sided marketplace platform for freelance work. The platform provides onboarding, role-based dashboards, gig listing and proposal workflows, direct messaging, and payments.

2.2 Product Functions

The system provides the following major functions:

User authentication and role-based access

Freelancer onboarding and profile management

Client onboarding and company profile management

Gig creation, listing, and browsing

Proposal submission and review

Conversation and messaging management

Payment creation, verification, and completion flow

Dashboard routing based on the signed-in user role

2.3 User Classes and Characteristics

Client

A client is a user who posts work, reviews proposals, accepts freelancers, and uses chat and payment features to manage projects.

Freelancer

A freelancer is a user who creates a professional profile, browses gigs, submits proposals, chats with clients, and receives payment for work.

Guest

A guest is an unauthenticated visitor who can view the landing page, about page, and public guidance pages, but cannot access protected workflows.

2.4 Operating Environment

Web browser on desktop or mobile

Next.js application server

Convex backend database and functions

Clerk authentication service

Razorpay payment gateway

2.5 Design and Implementation Constraints

The platform must support separate client and freelancer flows.

Protected actions require authenticated sessions.

Payment processing must use Razorpay checkout.

Chat and marketplace data must be stored through Convex.

The ER diagram is intentionally excluded from this SRS.

2.6 Assumptions and Dependencies

Users have access to modern browsers with JavaScript enabled.

Authentication is managed by Clerk.

Payment success depends on Razorpay availability and order verification.

Backend records are stored in Convex tables and accessed through Convex queries and mutations.

3. System Features

3.1 Authentication and Role Selection

Users sign in through Clerk and select one of two role paths when no role is assigned:

Freelancer onboarding

Client onboarding

Functional Requirements

The system shall allow users to sign in and sign out.

The system shall detect whether a user already has a role.

The system shall route unassigned users to onboarding selection.

The system shall route freelancers to freelancer-specific pages.

The system shall route clients to client-specific pages.

3.2 Freelancer Profile Management

Freelancers can create a professional profile containing personal and work-related information.

Functional Requirements

The system shall allow freelancers to enter full name, title, location, bio, and skills.

The system shall allow freelancers to upload a profile image.

The system shall store freelancer profile data in the backend.

The system shall allow freelancers to update profile information later.

3.3 Client Profile Management

Clients can create a company profile containing company details and a logo.

Functional Requirements

The system shall allow clients to enter company name, contact name, industry, website, and description.

The system shall allow clients to upload a company logo.

The system shall store client profile data in the backend.

The system shall allow clients to update company information later.

3.4 Gig Management

Clients can create gig records and freelancers can browse available gigs.

Functional Requirements

The system shall allow clients to create gigs.

The system shall store gig title, description, budget, category, skills, deadline, and status.

The system shall allow gig status values such as open, in progress, and completed.

The system shall display available gigs to freelancers.

3.5 Proposal Management

Freelancers submit proposals in response to gigs.

Functional Requirements

The system shall allow freelancers to submit a proposal for a gig.

The system shall store cover letter, price, estimated days, and status.

The system shall allow clients to review proposals.

The system shall support proposal statuses such as pending, accepted, and rejected.

3.6 Chat and Conversation Management

The platform supports direct communication between clients and freelancers.

Functional Requirements

The system shall create and list conversations.

The system shall store messages linked to a conversation.

The system shall allow users to send messages.

The system shall allow message and conversation deletion where permitted.

The system shall track notifications for unread messages and proposal-related updates.

3.7 Payment Processing

The platform supports secure project payments through Razorpay.

Functional Requirements

The system shall allow a user to enter a payment amount in INR.

The system shall create a Razorpay order through the backend.

The system shall open Razorpay checkout for payment completion.

The system shall verify payment signatures after checkout.

The system shall redirect the user to a success page after verification.

The system shall support completion of a gig when payment is confirmed.

3.8 Public Pages and Guidance

The platform provides public pages that explain the product and user journey.

Functional Requirements

The system shall display a public landing page.

The system shall display an about page describing the platform.

The system shall display a guide page that explains how clients and freelancers use the platform.

4. External Interface Requirements

4.1 User Interface Requirements

The interface shall be responsive for desktop and mobile screens.

The interface shall provide clear navigation between onboarding, dashboards, chat, and payment pages.

The interface shall provide loading states for asynchronous actions.

The interface shall provide visible feedback for save, send, and pay actions.

4.2 Software Interfaces

****Clerk:**** user authentication and session management

****Convex:**** queries, mutations, and database storage

****Razorpay:**** payment checkout and verification

4.3 Communication Interfaces

The frontend shall communicate with backend endpoints using HTTPS.

The payment page shall call order creation and verification endpoints.

Chat interactions shall use Convex queries and mutations.

5. Data Requirements

5.1 Core Data Entities

The system stores and processes the following logical data entities:

User account

Freelancer profile

Client profile

Gig

Proposal

Conversation

Message

Notification

5.2 Data Retention and Integrity

User role and profile data shall remain linked to the authenticated account.

Gigs shall retain their creator reference.

Proposals shall remain linked to a specific gig and freelancer.

Messages shall remain linked to a specific conversation.

Notification status shall indicate whether the item has been read.

6. Non-Functional Requirements

6.1 Performance

Pages should load within a reasonable time on standard broadband connections.

Dashboard and chat views should render loading indicators while data is fetched.

6.2 Security

Protected routes should require authentication.

Payment verification must occur server-side.

Sensitive profile and message data must only be accessible to authorized users.

6.3 Reliability

The system should handle absent or delayed backend data gracefully.

Payment flow should fail safely if checkout or verification fails.

6.4 Usability

The workflow should be easy to follow for both clients and freelancers.

The platform should present role choices clearly.

Forms should provide labels, placeholders, and feedback for input actions.

6.5 Maintainability

The system should keep routing, backend data, and payment logic separated by feature area.

The platform should use reusable components where appropriate.

6.6 Scalability

The architecture should support growth in users, gigs, conversations, and payments without requiring a redesign of the core workflow.

7. Use Case Specifications

7.1 Use Case: Freelancer Onboarding

****Actor:**** Freelancer

****Precondition:**** User is signed in and has selected the freelancer path.

****Main Flow:****

1. User opens freelancer onboarding.
2. User enters profile details.
3. User uploads a profile photo.
4. User saves the profile.
5. System stores the profile and redirects to the freelancer dashboard.

7.2 Use Case: Client Onboarding

****Actor:**** Client

****Precondition:**** User is signed in and has selected the client path.

****Main Flow:****

1. User opens client onboarding.
2. User enters company details.
3. User uploads a company logo.
4. User saves the profile.
5. System stores the profile and redirects to the client dashboard.

7.3 Use Case: Submit Proposal

****Actor:**** Freelancer

****Precondition:**** A gig is open and visible to the freelancer.

****Main Flow:****

1. Freelancer opens a gig.
2. Freelancer writes a cover letter.
3. Freelancer enters a proposed price and estimated days.
4. Freelancer submits the proposal.
5. System stores the proposal as pending.

7.4 Use Case: Accept Proposal and Chat

****Actor:**** Client

****Precondition:**** At least one proposal exists for a gig.

****Main Flow:****

1. Client reviews submitted proposals.
2. Client accepts one proposal.
3. System creates or updates the conversation context.
4. Both users can exchange messages in chat.

7.5 Use Case: Make Payment

****Actor:**** Client or project payer

****Precondition:**** Payment amount is entered and Razorpay is available.

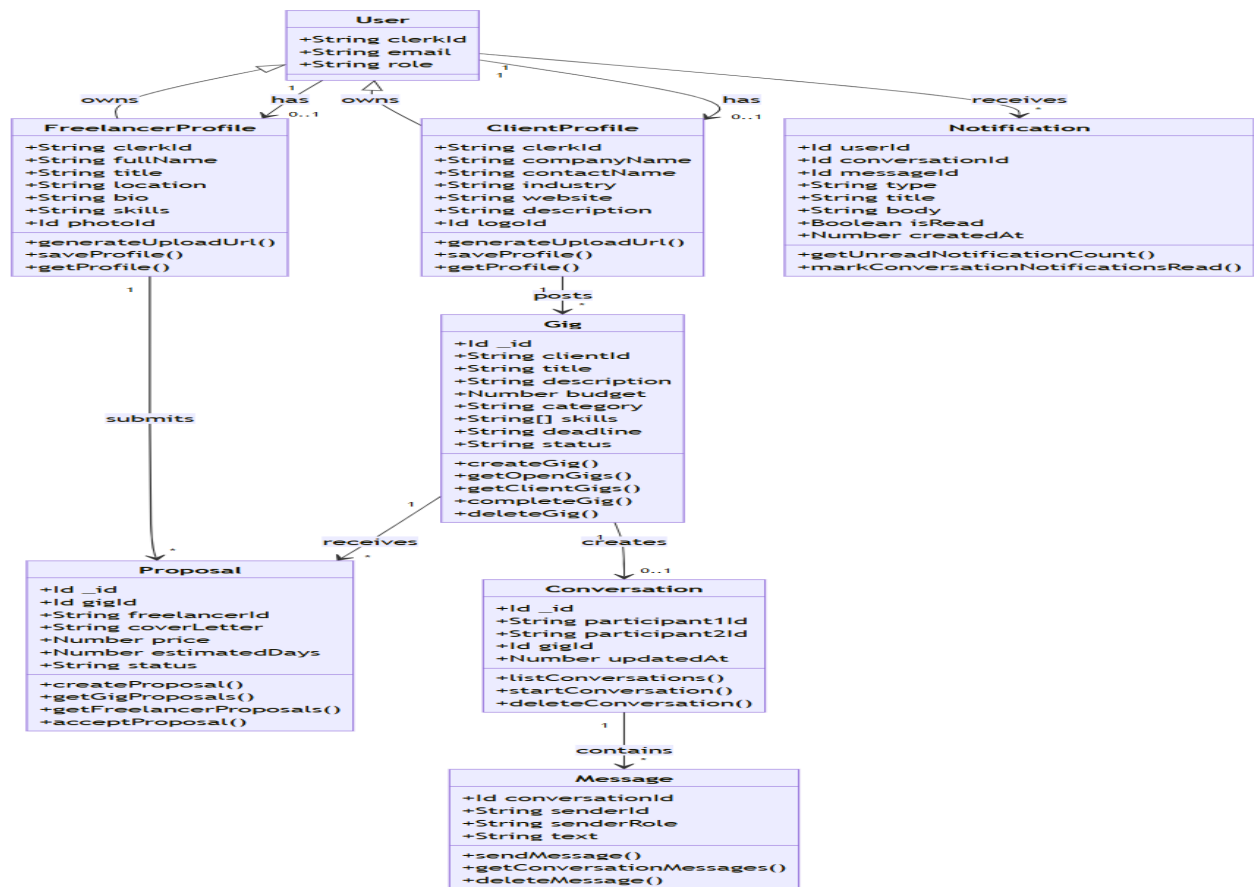
****Main Flow:****

1. User opens the payment page.
2. User enters an amount.

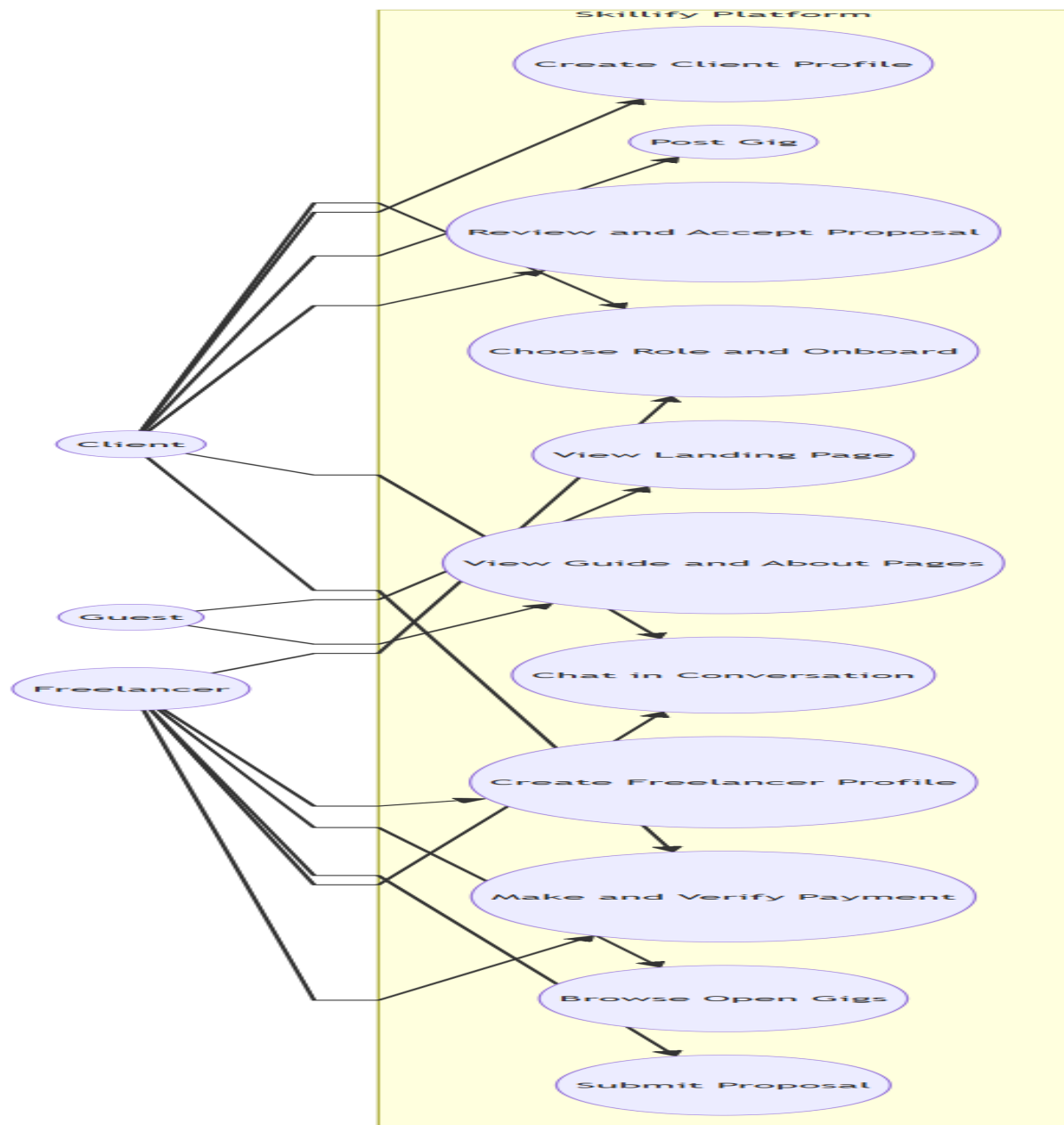
3. System creates a Razorpay order.
4. Razorpay checkout opens.
5. Payment is verified.
6. System shows success and optionally marks the gig complete.

8. System Diagrams

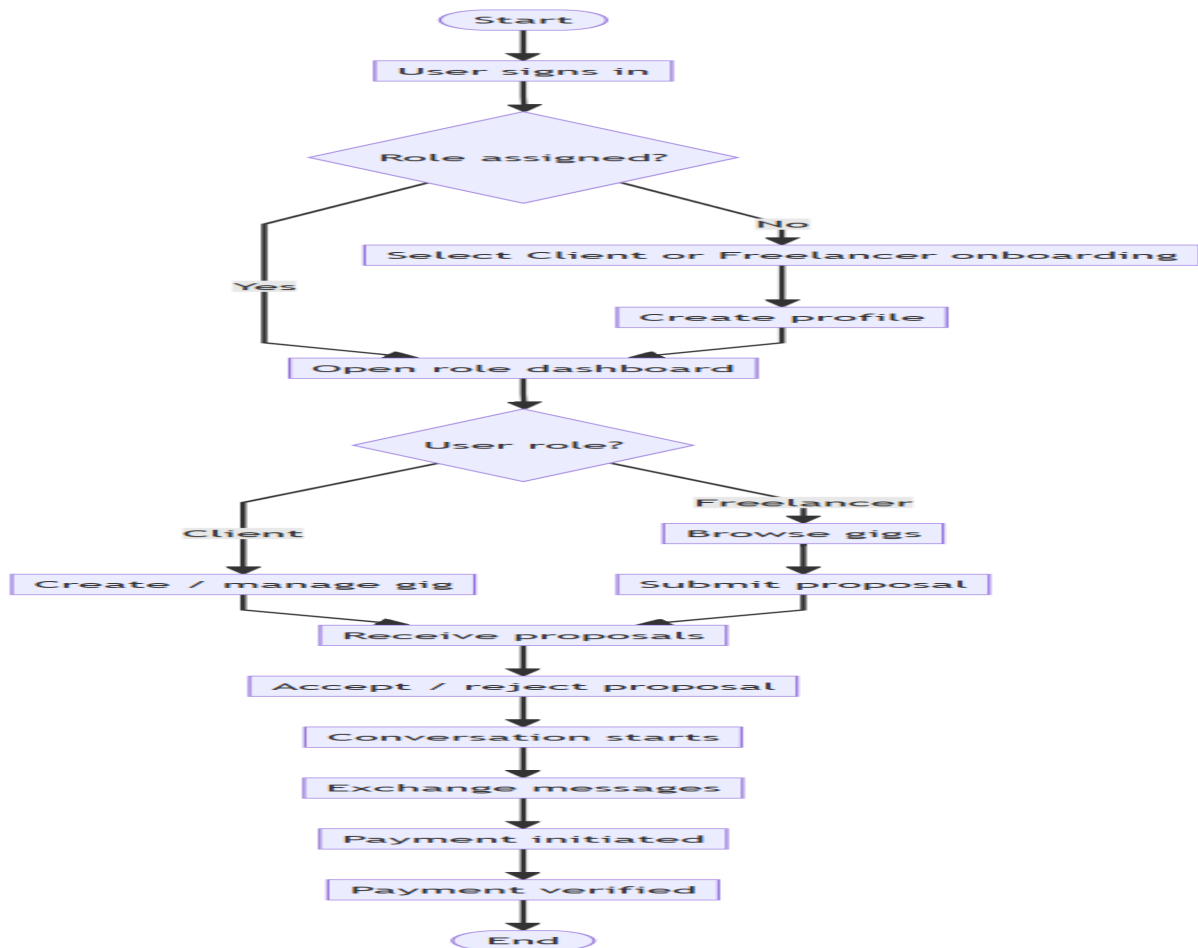
8.1 Context Diagram



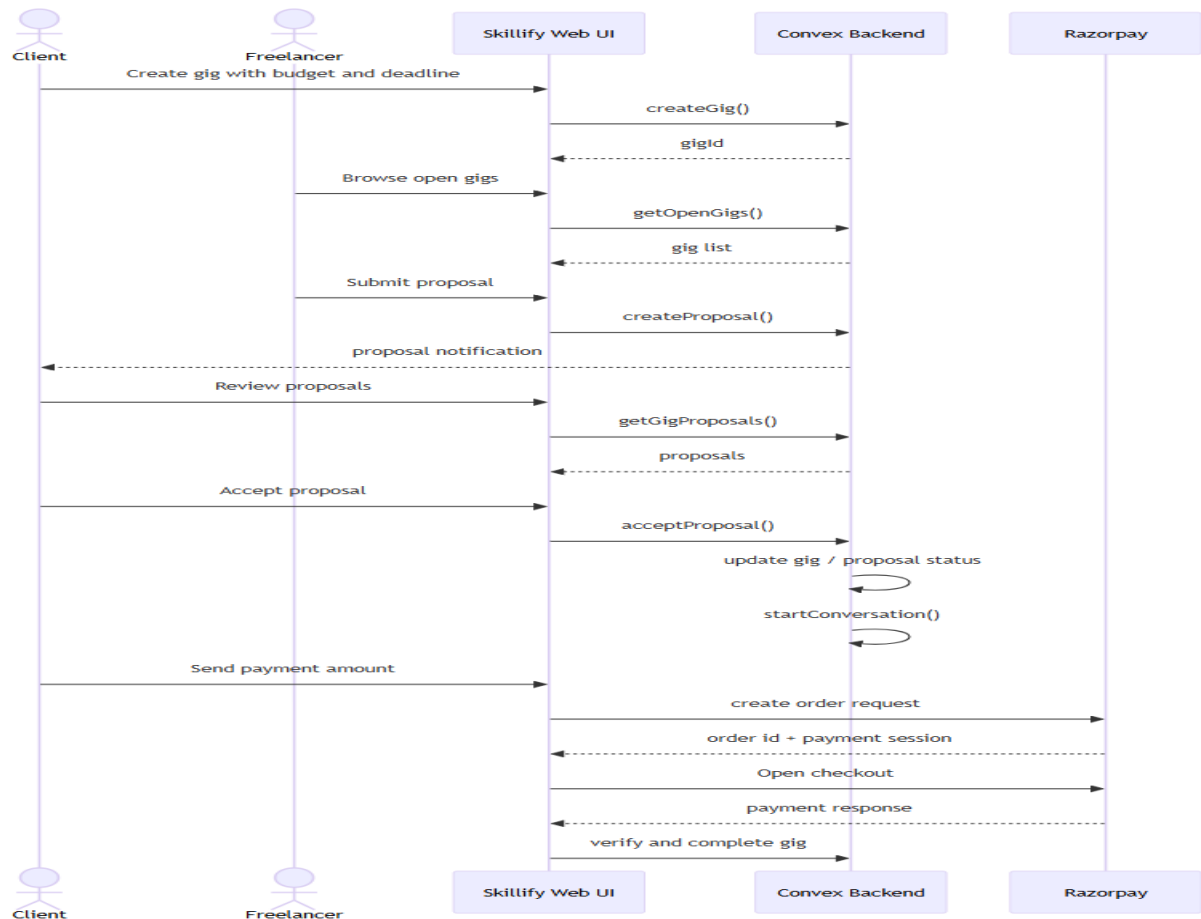
8.2 Use Case Diagram



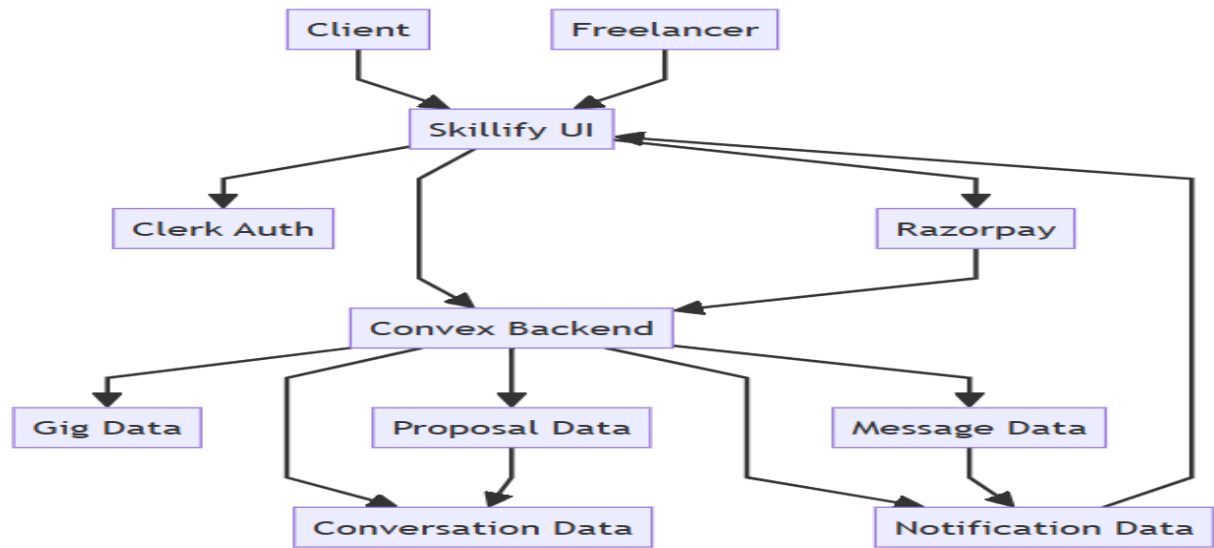
8.3 Activity Diagram



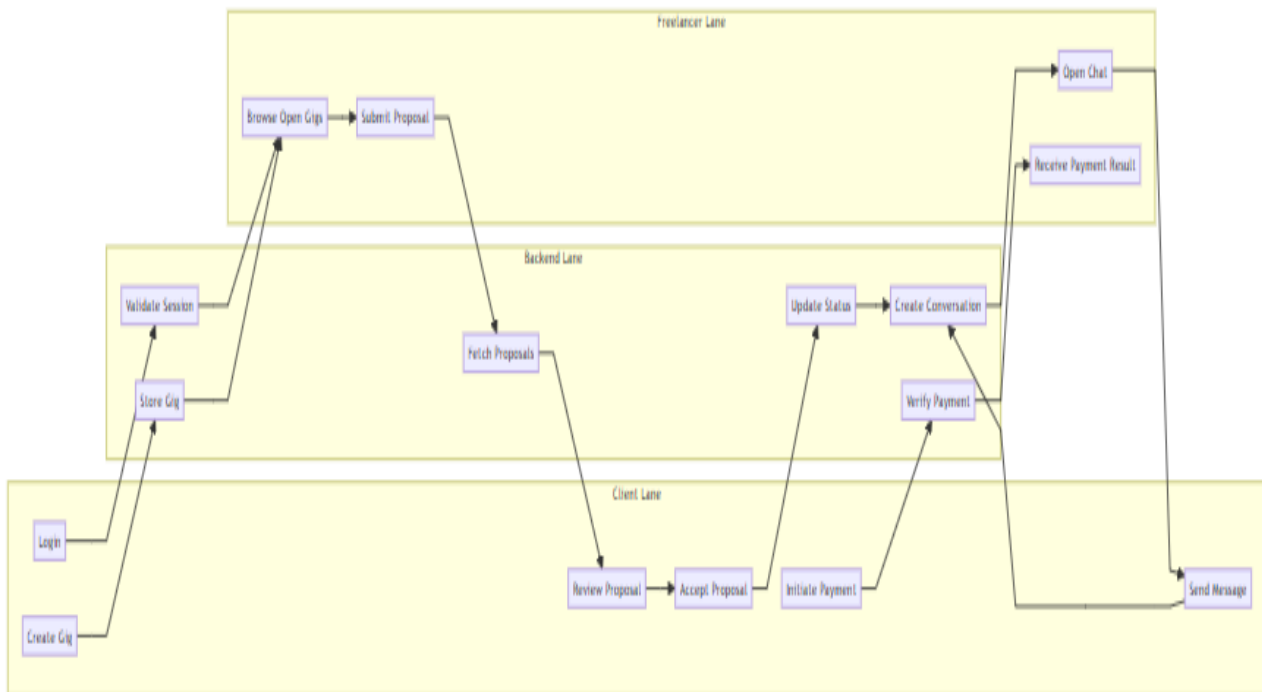
8.4 Sequence Diagram



8.5 Collaboration Diagram



8.6 Swimlane Diagram



9. Requirements Traceability Summary

Requirement Area	Supported Features
Authentication	Sign in, sign out, role detection, routing
Onboarding	Freelancer profile, client profile
Marketplace	Gig listing, proposal submission
Communication	Conversations, messages, notifications
Payments	Razorpay order creation, verification, success flow
Public Content	Landing page, about page, guide page

10. Appendices

10.1 Notes on Diagram Scope

This SRS includes context, use case, activity, sequence, collaboration, and swimlane diagrams. The ER diagram is intentionally omitted.

10.2 Editable Format

This document is provided in PDF format for easy sharing and submission. The editable source is stored in Markdown and Word formats in the workspace.